



"Ss. Cyril and Methodius" University in Skopje
**FACULTY OF COMPUTER
SCIENCE AND ENGINEERING**

Smart Travel Planner

Service-Oriented Architecture Project Specification



Smart Travel
— **PLANNER** —

Authors:

Nikola Sarafimov
Klaudija Stamenova

Mentors:

Prof. Dr. Riste Stojanov
Asst. Elena Atanasoska, M.Sc

Year: 2026

1. Introduction

Smart Travel Planner is a service-oriented microservice application designed to support travel planning through trip management, destination-based recommendations, saved places, cost estimation and tool-based access through an MCP Server. The system allows users to create trips, search for hotels, restaurants and attractions, save recommendations to a specific trip, and estimate the total cost of the selected travel plan.

The project is developed as an independent domain-specific application and is not based on FINKI laboratory service examples. The selected domain is travel planning, which naturally supports decomposition into multiple bounded contexts and independent services. This makes it suitable for demonstrating a complete Service-Oriented Architecture implementation.

The main goal of the project is not only to implement business functionality, but also to demonstrate key SOA concepts such as microservice decomposition, API Gateway routing, service discovery, authentication and authorization, synchronous and asynchronous service communication, external API integration, contract testing, Dockerized deployment and MCP Server integration.

2. Project Goals

The main goals of the Smart Travel Planner system are:

- To provide a functional travel planning application with trip management.
- To provide destination-based recommendations for attractions, restaurants and hotels.
- To allow users to save recommendations to planned trips.
- To estimate the total cost of saved recommendations for a trip.
- To integrate an external API for live recommendation data.
- To demonstrate microservice decomposition using separate bounded contexts.
- To secure protected API endpoints using Keycloak and JWT tokens.
- To use an API Gateway as the centralized entry point.
- To use Consul for service registration and service discovery.
- To demonstrate synchronous communication with OpenFeign.
- To demonstrate asynchronous communication with Kafka.
- To provide Pact-based consumer-driven contract testing.
- To expose selected backend functionality through an MCP Server.
- To run the entire system through Docker Compose.
- To provide a React frontend for functional testing and presentation.

3. High-Level System Architecture

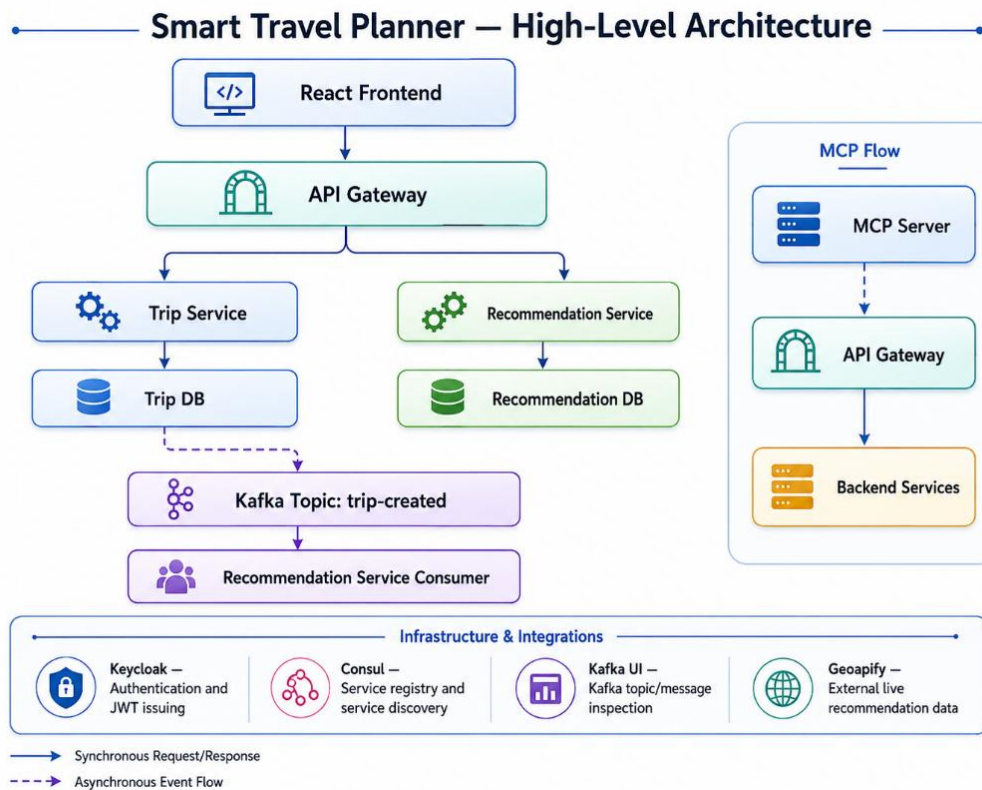
The system is composed of several independent services and infrastructure components. The frontend communicates with the backend through the API Gateway. The backend is divided into separate services, each responsible for its own bounded context and database.

The main components are:

- React Frontend
- API Gateway
- Trip Service
- Recommendation Service
- MCP Server
- Trip PostgreSQL Database
- Recommendation PostgreSQL Database
- Kafka

- Kafka UI
- Consul
- Keycloak
- Geoapify Places API

The high-level architecture is shown below:



The frontend does not directly call the Trip Service or the Recommendation Service. All business requests are routed through the API Gateway. This follows a common microservice architecture pattern where the gateway acts as the single entry point for clients.

4. Microservice Decomposition and Bounded Contexts

The system is divided into multiple bounded contexts according to the principles of Domain-Driven Design. Each bounded context has its own responsibility, data model and service boundary.

4.1 Trip Bounded Context

The Trip Service is responsible for the trip planning domain. It manages trips created by users and stores trip-related information such as destination, travel period, budget, currency and status.

Main responsibilities:

- Creating a trip.
- Listing all trips.
- Listing trips by user.
- Getting a trip by ID.
- Updating trip information.
- Deleting a trip.
- Publishing a Kafka event when a trip is created.
- Calling the Recommendation Service through Feign.

- Requesting estimated cost for a trip.

The main entity in this bounded context is Trip.

Important fields:

- id
- userId
- destination
- startDate
- endDate
- budget
- currency
- status
- createdAt

Trip statuses include:

- PLANNED
- ACTIVE
- COMPLETED
- CANCELLED

4.2 Recommendation Bounded Context

The Recommendation Service is responsible for travel recommendations. It manages recommendations for destinations, including attractions, hotels and restaurants. It also supports saved recommendations and trip cost estimation.

Main responsibilities:

- Returning recommendations by destination.
- Returning recommendations by type.
- Returning hotels by budget.
- Returning restaurants and attractions.
- Loading local seed data.
- Calling Geoapify Places API as a live fallback.
- Persisting live external recommendations.
- Avoiding duplicate live recommendations using externalPlaceId.
- Saving recommendations to trips.
- Loading saved recommendations.
- Estimating total cost of saved recommendations.
- Consuming Kafka trip-created events.

The main entities in this bounded context are:

- Recommendation
- SavedRecommendation

The Recommendation entity contains fields such as:

- id
- destination
- name
- type
- description

- estimatedPrice
- rating
- source
- externalPlaceId

The SavedRecommendation entity connects a recommendation to a trip and stores when it was saved.

4.3 MCP Integration Context

The MCP Server is responsible for exposing selected Smart Travel Planner functionality as MCP tools. It does not directly access databases. Instead, it communicates with backend services through the API Gateway.

Main responsibilities:

- Exposing MCP tools.
- Calling backend APIs through the API Gateway.
- Using Keycloak client credentials flow to obtain a service token.
- Providing REST helper endpoints for frontend MCP testing.
- Supporting MCP Inspector testing.

Exposed MCP tools:

- recommend_places
- get_trip_details
- get_saved_attractions
- estimate_trip_cost

5. API Gateway

The API Gateway is implemented as a centralized entry point for backend communication. It routes requests from the frontend to the correct backend service and enforces security for protected routes.

Main responsibilities:

- Routing trip requests to Trip Service.
- Routing recommendation requests to Recommendation Service.
- Routing MCP requests when needed.
- Validating JWT tokens issued by Keycloak.
- Using Consul service names instead of hardcoded internal URLs.

Gateway routes:

- /api/trips/** -> trip-service
- /api/recommendations/** -> recommendation-service
- /mcp/** -> mcp-server

The API Gateway allows the frontend to communicate with the system through a single backend entry point: <http://localhost:8080>

This improves separation between the frontend and the internal microservice structure.

6. Service Discovery with Consul

Consul is used as the service registry and discovery mechanism. Backend services register themselves in Consul and can be discovered by service name.

Registered services:

- api-gateway
- trip-service
- recommendation-service
- mcp-server

Consul is available at: <http://localhost:8500>

This makes it possible for the API Gateway and Feign clients to use logical service names instead of fixed hostnames. For example, the Trip Service can call the Recommendation Service using the service name: recommendation-service

This demonstrates service discovery, which is an important concept in service-oriented and microservice architectures.

7. Inter-Service Communication

The system demonstrates several types of communication between services.

7.1 Client-to-Service Communication Through API Gateway

The React frontend communicates with backend services through the API Gateway.

Example flow:

- Frontend -> API Gateway -> Trip Service
- Frontend -> API Gateway -> Recommendation Service

This is used for actions such as:

- Creating trips.
- Loading trips.
- Searching recommendations.
- Saving recommendations.
- Estimating costs.

7.2 Synchronous Communication with OpenFeign

The Trip Service communicates synchronously with the Recommendation Service using OpenFeign.

Feign is used for:

- Loading recommendations for a trip destination.
- Estimating the cost of saved recommendations for a trip.

Example flow: Trip Service -> Recommendation Service

The Trip Service forwards the JWT Authorization header when making Feign calls. This allows secure inter-service communication and ensures that protected endpoints remain secured even when accessed internally.

7.3 Asynchronous Communication with Kafka

Kafka is used for asynchronous event-based communication.

When a new trip is created, the Trip Service publishes a trip-created event to Kafka.

Event flow: Trip Service -> Kafka topic: trip-created -> Recommendation Service

The Recommendation Service consumes the event and logs the received trip information. This demonstrates asynchronous communication between services.

Kafka UI is available at: <http://localhost:8085>

The expected topic is: trip-created

7.4 MCP Server Communication

The MCP Server communicates with the backend through the API Gateway. It first obtains a Keycloak service token using the client_credentials flow and then calls protected backend endpoints with a Bearer token.

MCP communication flow:

- MCP Server -> Keycloak token endpoint
- MCP Server -> API Gateway -> Backend Services

This ensures that MCP tools do not bypass the security model of the system.

8. Security and Authentication

Keycloak is used as the identity provider for authentication and authorization. The system uses JWT access tokens to secure protected API endpoints.

Keycloak runs on: <http://localhost:8086>

The configured realm is: smart-travel

The system uses two Keycloak clients:

8.1 Frontend Client

The frontend uses a public Keycloak client: smart-travel-client

This client is used for user login through the browser. The demo user is:

- Username: demo-user
- Password: demo-pass

After login, the frontend receives a JWT access token and sends it in the Authorization header when making API requests: Authorization: Bearer <access_token>

8.2 MCP Confidential Client

The MCP Server uses a confidential Keycloak client: mcp-server-client

This client uses the client_credentials flow to obtain a service token. The client secret is not hardcoded in the code. It is provided through the .env file.

8.3 Protected Endpoints

The API Gateway, Trip Service and Recommendation Service validate JWT tokens. Protected API endpoints return 401 Unauthorized when called without a valid token.

Health endpoints are public because they are needed by Docker, Consul and the frontend health dashboard.

9. Data Persistence and Database Separation

The system uses separate PostgreSQL databases for different bounded contexts. This follows the database-per-service pattern.

9.1 Trip Database

The Trip Service uses: trip_db

Main table: trips

The table stores trip-related data such as:

- trip ID;
- user ID;
- destination;
- start date;
- end date;
- budget;
- currency;
- status;
- creation timestamp.

9.2 Recommendation Database

The Recommendation Service uses: recommendation_db

Main tables: recommendations, saved_recommendations

The recommendations table stores both local seed recommendations and live Geoapify recommendations.

The saved_recommendations table stores recommendations saved to a trip.

The externalPlaceId field is used to prevent duplicate records when live recommendations are loaded from the external API multiple times.

This database separation ensures that the Trip Service and Recommendation Service do not share the same database schema and remain independently deployable.

10. External API Integration

The Recommendation Service integrates with Geoapify Places API to provide live recommendation data.

External provider: Geoapify Places API

The external API is used when local seed data does not exist for the requested destination and recommendation type.

The implemented strategy is:

1. Search local seed recommendations first.
2. If seed recommendations exist, return them.
3. If no seed recommendations exist, call Geoapify Places API.
4. Map Geoapify results to Recommendation entities.
5. Persist live results in the database.
6. Use externalPlaceId to avoid duplicate database records.

This approach provides stable demo data for known destinations such as Paris, while also supporting live external data for other destinations such as Rome, Berlin, Skopje or Ohrid.

The external API status can be checked through:
GET /api/recommendations/external/status

This endpoint returns information about:

- provider name;
- whether the external API is enabled;
- whether the API key is configured;
- fallback strategy;
- deduplication key;
- backend flow.

11. MCP Server

The MCP Server is implemented to expose Smart Travel Planner functionality through MCP tools. This allows selected backend functionality to be accessed in a tool-based way.

The MCP Server exposes the following tools:

- recommend_places (Returns recommendations for a destination. It can filter by type and limit the number of results.)
- get_trip_details (Returns details for a specific trip by trip ID.)
- get_saved_attractions (Returns saved attractions for a specific trip.)
- estimate_trip_cost (Returns the estimated total cost of saved recommendations for a trip.)

11.1 MCP Testing

The MCP Server can be tested in two ways.

First, through the frontend MCP Tester page, which calls REST helper endpoints:

- GET /api/mcp-test/recommend-places
- GET /api/mcp-test/trip/{tripId}
- GET /api/mcp-test/saved-attractions
- GET /api/mcp-test/estimate

Second, through MCP Inspector: `npx @modelcontextprotocol/inspector`

MCP Inspector configuration:

- Transport: Streamable HTTP
- URL: `http://localhost:8087/mcp`

This confirms that the MCP tools are exposed and available for external tool-based interaction.

12. Frontend Application

The frontend is implemented with React and Vite and is served through Nginx in Docker.

The frontend is available at: <http://localhost:3000>

Frontend pages:

- Dashboard
- Trips

- Recommendations
- Saved & Cost
- MCP Tester
- SOA Proof

Main frontend responsibilities:

- Keycloak login.
- Display system health.
- Create and manage trips.
- Search seed and live recommendations.
- Save recommendations to trips.
- Estimate trip costs.
- Run Feign communication tests.
- Run MCP helper endpoint tests.
- Display SOA requirement coverage.
- Provide links to Consul, Kafka UI and Keycloak.

The frontend displays currency as EURO, while the backend internally stores the currency code as EUR.

13. Dockerized Deployment

The entire system is Dockerized and can be started using Docker Compose.

Start command: `docker compose up -d --build`

Docker Compose starts the following components:

- frontend;
- api-gateway;
- trip-service;
- recommendation-service;
- mcp-server;
- trip-db;
- recommendation-db;
- kafka;
- kafka-ui;
- consul;
- keycloak.

Each backend service has its own Dockerfile. The frontend is built using Node and served using Nginx. Services communicate through the Docker network using container service names.

Environment variables are provided through the root .env file:

`MCP_CLIENT_SECRET=...`

`GEOAPIFY_API_KEY=...`

14. Testing and Verification

The project includes several forms of testing and verification.

14.1 HTTP Tests

The docs/docker-tests.http file contains a complete test flow for the Dockerized system.

It verifies:

- Keycloak token generation;
- public health endpoints;
- protected endpoints without token;
- protected endpoints with token;
- seed recommendations;
- live Geoapify recommendations;
- external API status;
- trip creation;
- trip loading;
- trip updating;
- Feign communication;
- saving recommendations;
- cost estimation;
- MCP REST helper endpoints.

14.2 Keycloak Security Test

The docs/keycloak-test.http file verifies that protected endpoints return 401 Unauthorized without a token and work correctly with a valid Keycloak token.

14.3 Integration Tests

The docs/integration-tests.http file provides a shorter integration flow that checks the main backend functionality through the API Gateway.

14.4 Pact Contract Testing

Pact is used for consumer-driven contract testing between the Trip Service and the Recommendation Service.

The Trip Service acts as the consumer.

The Recommendation Service acts as the provider.

14.5 Manual Verification

Manual verification includes:

- checking that all containers are running with docker compose ps;
- logging in through Keycloak;
- creating a trip;
- checking Kafka UI for the trip-created topic;
- checking Consul UI for registered services;
- searching recommendations;
- saving recommendations;
- estimating cost;
- testing MCP tools;
- checking the frontend SOA Proof page.

15. Conclusion

Smart Travel Planner demonstrates a complete service-oriented architecture implemented through independent microservices and supporting infrastructure. The system includes a clear domain decomposition, centralized gateway routing, service discovery, secure authentication, synchronous and asynchronous communication, database separation, external API integration, contract testing, Dockerized deployment and MCP Server functionality.

The Trip Service and Recommendation Service represent separate bounded contexts, while the API Gateway, Consul, Keycloak, Kafka and MCP Server provide the necessary infrastructure for a realistic distributed system. The React frontend provides a practical interface for testing and presenting all implemented functionalities.

Overall, the project satisfies the main requirements of a Service-Oriented Architecture system and provides a complete working example of a Dockerized microservice application.